Proceedings of the 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS-2025)
IEEE Xplore Part Number: CFP25DN7-ART; ISBN: 979-8-3315-3884-2

2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS) | 979-8-3315-3884-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSSAS66150.2025.11081258

JudgmentGraph: Benchmarking Legal Knowledge Graph Construction from Supreme Court Rulings

Ajay Mukund S
Department of Computer Science and Engineering
College of Engineering Guindy, Anna University
Chennai, India
ajaymukund1998@gmail.com

Dr. K. S. Easwararkumar
Department of Computer Science and Engineering
College of Engineering Guindy, Anna University
Chennai, India
easwara@annauniv.edu

***Abstract*—Legal judgments are rich in factual, procedural, and normative content, yet are typically presented in unstructured natural language, limiting their accessibility for automated reasoning and retrieval. This paper proposes a comprehensive and ontology-guided pipeline for constructing a Legal Knowledge Graph from Indian Supreme Court judgments. The pipeline integrates a fine-grained Legal Named Entity Recognition system with a formally defined Legal Ontology to generate semantically valid triples. A multi-stage relation extraction module leverages syntactic cues, proximity heuristics, and logical constraints to refine relation candidates. The resulting triples are serialized into RDF and visualized using WebVOWL. Empirical evaluation on a manually annotated gold-standard corpus demonstrates strong performance with a Precision of 81.4%, Recall of 88.4%, and an F1-score of 84.7%. A qualitative case study on the *Lalita Kumari v. Government of Uttar Pradesh* ruling illustrates the interpretability and domain-alignment of the extracted graph. This work contributes the first benchmarked pipeline for legal knowledge graph construction in the Indian legal domain and lays the foundation for downstream applications in legal information retrieval, reasoning, and decision support.**

***Index Terms*—Legal Knowledge Graph, Legal NLP, Named Entity Recognition, Ontology, Relation Extraction, Triple Generation**

I. Introduction

Legal judgments are repositories of statutory interpretation, procedural history, and judicial reasoning. Yet, despite their rich informational content, these documents remain largely unstructured, typically composed in natural language with considerable variation in format, expression, and structure. Indian Supreme Court judgments, in particular, encompass complex references to litigants, statutes, legal counsel, courts, and precedents—elements critical to legal understanding but embedded in free-form textual narratives that challenge automated processing.

With the increasing integration of Artificial Intelligence (AI) into legal research and decision-making workflows, there is a growing demand for structured and machine-interpretable representations of legal texts. Legal Knowledge Graphs (KGs), which represent legal entities and their semantic relationships as interconnected triples, offer a promising foundation for structured legal analytics. These representations facilitate advanced legal information retrieval, semantic search, precedent linkage, argument extraction, and legal trend analysis.

However, constructing high-quality Legal KGs from court rulings remains a non-trivial challenge. Legal relationships are often implicit, span multiple sentences or sections, and rely heavily on domain expertise. Furthermore, the inherent variability in legal writing—across judges, courts, and case types—complicates the standardization of entity recognition and relation extraction. Existing approaches, particularly those developed for Western legal systems, often lack compatibility with Indian legal discourse and fail to offer benchmarked evaluations or reusable pipelines tailored to this domain.

To address these limitations, this work proposes a comprehensive and ontology-constrained pipeline for constructing Legal Knowledge Graphs from Indian Supreme Court judgments. The framework builds on two previously developed components—namely, a domain-specific **Legal Named Entity Recognition (NER)** model and a formally specified **Legal Ontology**. The Legal NER system [1] identifies fine-grained entities including *Petitioner*, *Respondent*, *Judge*, *Provision*, *Statute*, *Precedent*, and others, while the Legal Ontology defines over 25 object properties to model core legal relationships.

The key contributions of this work are as follows:

1) **Pipeline Proposal**: An end-to-end framework is presented that converts unstructured Supreme Court judgments into structured RDF triples by integrating Legal NER with ontology-constrained relation extraction.
2) **Relation Mapping Algorithms**: A combination of heuristic and ontology-guided algorithms is developed to infer legal relationships between extracted entities, leveraging domain-range constraints encoded in the ontology.
3) **Graph Construction and Evaluation**: A Legal KG is constructed from a curated judgment corpus and the quality of relation extraction is evaluated through manual annotation. Graph-level statistics such as node distributions and connectivity are also reported.
4) **Open and Visualizable Resource**: The resulting RDF-based Legal KG is visualized using **WebVOWL** [2], and the KG itself is made available to the research community for reproducibility and reuse.

This paper offers the first benchmarked pipeline for Legal

Knowledge Graph construction tailored to the Indian Supreme Court, providing foundational infrastructure for future research in structured legal data modeling, reasoning, and legal AI.

II. Related Work

Legal Knowledge Graphs (LKGs) are gaining traction as a means to structure and interlink legal information, enabling enhanced retrieval, reasoning, and analytical capabilities across legal corpora. They typically encode entities such as laws, cases, and actors, along with their semantic relations, as machine-readable triples.

A. Legal Knowledge Graph Construction

Recent approaches to LKG construction rely on combining entity and relation extraction with ontology-driven triple generation. Jain et al. [3] proposed a pipeline tailored to Indian legal texts, integrating entity recognition and ontology-based relation typing. **Lex2KG**, introduced by Abdurahman et al. [4], enables automatic conversion of legal documents, including PDFs, into RDF-based KGs that capture inter-document links such as citations and amendments. Vuong et al. [5] demonstrated a heterogeneous graph framework for Vietnamese legal cases, incorporating multiple legal entities to enhance semantic modeling.

Knowledge-enhanced models have also shown promise. Li et al. [6] embedded legal knowledge into large language models via prefix-tuning to improve relation extraction in the construction of a Chinese Legal KG. Similarly, Bi et al. [7] employed graph embeddings to assess document similarity by capturing court, charge, and case-level connections in Chinese law.

B. Legal Ontologies

Ontologies provide the semantic backbone for LKGs, ensuring consistency in representation. Prominent examples include **LKIF-Core** [8], which models core legal concepts and norms; **LegalRuleML** [9], an XML-based formalism for representing legal rules and logic; and **ALeG**, an OWL ontology modeling European legal structures. The **Lynx** platform [10] integrates such ontologies to support knowledge-based services for legal document processing and enrichment.

C. Entity and Relation Extraction in Legal NLP

Legal NLP research has extensively addressed named entity and relation extraction. Sovrano et al. [11] developed a KG-based question answering framework, while Dhani et al. [12] applied LKGs to similar case recommendation. Our previously published Legal NER model, optimized for Indian legal discourse, identifies granular entity types such as *Judge*, *Precedent*, and *Provision* with high accuracy. Complementing this, our Legal Ontology defines over 25 relations including `representedByLawyer`, `citesPrecedent`, and `refersToProvision`, enabling domain-informed graph construction.

D. Gaps in Benchmarking for Indian Legal KGs

Despite progress in legal KG construction globally, the Indian legal domain lacks a publicly benchmarked, end-to-end system that integrates domain-specific NER, ontology-constrained relation extraction, and graph-level evaluation. Existing systems primarily evaluate extraction accuracy [13] [14], with limited focus on graph structural analysis or benchmarking against manually annotated triples. Our work addresses this gap by proposing a reproducible pipeline and evaluating both extraction and graph properties for Indian Supreme Court data.

III. Background: Legal NER and Ontology

The Legal Knowledge Graph construction pipeline proposed in this work builds upon two foundational components developed and communicated in prior research: a domain-specific Legal Named Entity Recognition (NER) model and a formally structured Legal Ontology. This section briefly outlines these components, which serve as essential prerequisites for the graph construction methodology described in subsequent sections.

A. Legal Named Entity Recognition

Our Legal NER system is trained to recognize and classify fine-grained legal entities commonly found in Indian Supreme Court judgments. The entity schema is specifically designed to capture the complex ecosystem of legal actors, institutional references, and normative elements. It includes the following types: *Petitioner*, *Respondent*, *Judge*, *Lawyer*, *Witness*, *Other_Person*, *Provision*, *Statute*, *Precedent*, *Court*, *Organization*, *Geopolitical Entity (GPE)*, *Date*, and *Case Number*. The Legal NER model used in this work is based on a **RoBERTa-large** architecture, enhanced with a **Dispersed Hierarchical Attention (DHA)** [15] mechanism and a Conditional Random Field (CRF) layer for sequence labeling. Fine-tuned on a manually annotated corpus of Indian Supreme Court judgments, the model achieves a state-of-the-art macro-averaged F1 score of 93.70, outperforming prior benchmarks by 2.6 points. The architecture is specifically designed to capture the nuanced structure and domain-specific terminology of Indian legal discourse, ensuring high precision and recall across entity types. To further enhance consistency in entity linking, a post-processing pipeline is employed, incorporating domain-informed coreference resolution techniques for handling precedent and statute-provision mentions. A detailed account of the model architecture, training procedure, and evaluation metrics is available in our currently submitted work to the Artificial Intelligence and Law journal [1].

B. Legal Ontology Specification

The Legal Ontology formalizes the semantic structure of the domain by defining classes for legal entities and object properties that encode their interrelations. Developed

[1]Legal NER dataset: https://github.com/Legal-NLP-EkStep/legal_NER

using OWL 2 and managed via the Protégé [16] ontology editor, the ontology models legal roles (e.g., *Judge*, *Lawyer*), legal documents (e.g., *Statute*, *Precedent*), and legal institutions (e.g., *Court*, *Organization*) within a coherent class hierarchy. A total of 28 object properties (relations) are defined in the ontology, each equipped with explicit domain and range constraints. These include, for instance, `representedByLawyer` (linking a party to its legal representative), `citesPrecedent` (linking a judgment to a precedent it references), and `refersToProvision` (associating a judgment with a statutory provision). The complete list of defined relations, along with their formal semantics, is available upon request.

To support interpretation and reuse, the ontology has been exported as an RDF [17] file and rendered using WebVOWL for interactive visualization. Both the ontology and its visual interface are made publicly available[2]. These components form the semantic and computational foundation for the relation extraction and triple construction process presented in the next section.

IV. Methodology

This section details the construction pipeline for transforming unstructured Supreme Court judgments into a structured Legal Knowledge Graph (LKG). The process leverages entities extracted by a pre-trained Legal NER model and relations defined by a formally specified Legal Ontology. Our methodology is divided into four main stages: entity extraction, ontology-constrained triple generation, relation disambiguation and pruning, and RDF-based graph serialization.

A. Entity Extraction

The input to our pipeline is a raw judgment document from the Indian Supreme Court. We apply our Legal NER system (described in Section III-A) to identify and classify legal entities such as *Petitioner*, *Respondent*, *Judge*, *Statute*, *Provision*, *Court*, and others. Each extracted entity is tagged with its corresponding class label from the Legal Ontology. The NER output forms the foundational set of nodes from which semantic triples are derived.

B. Triple Generation via Ontology Constraints

In this phase, we generate candidate triples of the form `<Subject, Predicate, Object>` by systematically pairing the recognized entities according to the object properties (relations) defined in our Legal Ontology. Each object property specifies domain and range constraints, which are used to filter semantically invalid combinations. For instance, the property `representedByLawyer` permits only entities of type *Petitioner* or *Respondent* as subjects, and a *Lawyer* as the object.

Let $\mathcal{E} = e_1, e_2, ..., e_n$ be the set of NER-labeled entities in a judgment, and let $\mathcal{R}$ be the set of ontology-defined object properties. For each (e_i, e_j) pair, we identify all $r \in \mathcal{R}$ such that:

$$\text{type}(e_i) = \text{domain}(r) \quad \text{and} \quad \text{type}(e_j) = \text{range}(r) \tag{1}$$

All such valid triples $\langle e_i, r, e_j \rangle$ are added to a candidate triple set $\mathcal{T}_{\text{cand}}$. This approach yields a maximally recall-oriented set of triples that is constrained by ontological semantics.

C. Mathematical Notation and Triple Construction

Let $\mathcal{E} = \{e_1, e_2, \ldots, e_n\}$ be the set of legal entities extracted from a given judgment using the Legal NER model, where each entity e_i is associated with a type label $\tau(e_i) \in \mathcal{C}$, and $\mathcal{C}$ is the set of ontology-defined legal classes (e.g., *Petitioner*, *Judge*, *Statute*).

Let $\mathcal{R}$ denote the set of object properties (relations) defined in the Legal Ontology. Each relation $r \in \mathcal{R}$ has a domain and range:

$$r : \text{domain}(r) \rightarrow \text{range}(r)$$

We define the candidate triple space $\mathcal{T}_{\text{cand}}$ as:

$$\mathcal{T}_{\text{cand}} = \left\{ \langle e_i, r, e_j \rangle \;\middle|\; \begin{array}{l} e_i, e_j \in \mathcal{E},\ r \in \mathcal{R}, \\ \tau(e_i) = \text{domain}(r),\ \tau(e_j) = \text{range}(r) \end{array} \right\}$$

This formulation ensures that only those triples are retained for which the subject-object entity types comply with the ontology's domain-range constraints.

Each candidate triple $\langle e_i, r, e_j \rangle$ is subsequently refined via the multi-stage relation disambiguation process detailed in the following subsections.

D. Relation Disambiguation and Filtering

While $\mathcal{T}_{\text{cand}}$ captures all ontology-compliant triples, not all of them accurately reflect the specific legal context of a judgment. To ensure high-quality relation extraction, we employ a multi-stage disambiguation and filtering pipeline comprising three sequential modules: (1) syntactic and lexical cue-based extraction, (2) proximity-based ranking, and (3) mutual exclusivity resolution. Each module progressively refines the triple set by incorporating increasingly precise heuristics grounded in legal discourse patterns.

1) ***Syntactic and Lexical Cue Matching****:* The first stage leverages explicit linguistic signals—verbs, phrases, and prepositions—to identify candidate relations between named entities. For each sentence in the judgment, we iterate over all entity pairs (e_i, e_j) that satisfy the domain and range constraints of an ontology-defined relation r. If any trigger pattern $p \in \mathcal{P}[r]$ is matched within the sentence (using regular expressions or dependency patterns), a corresponding triple $\langle e_i, r, e_j \rangle$ is added to the initial triple set $\mathcal{T}_{\text{lex}}$. This process is detailed in Algorithm 1.

[2]Legal Ontology RDF File: https://ajaymukunds.github.io/LegalOntology/

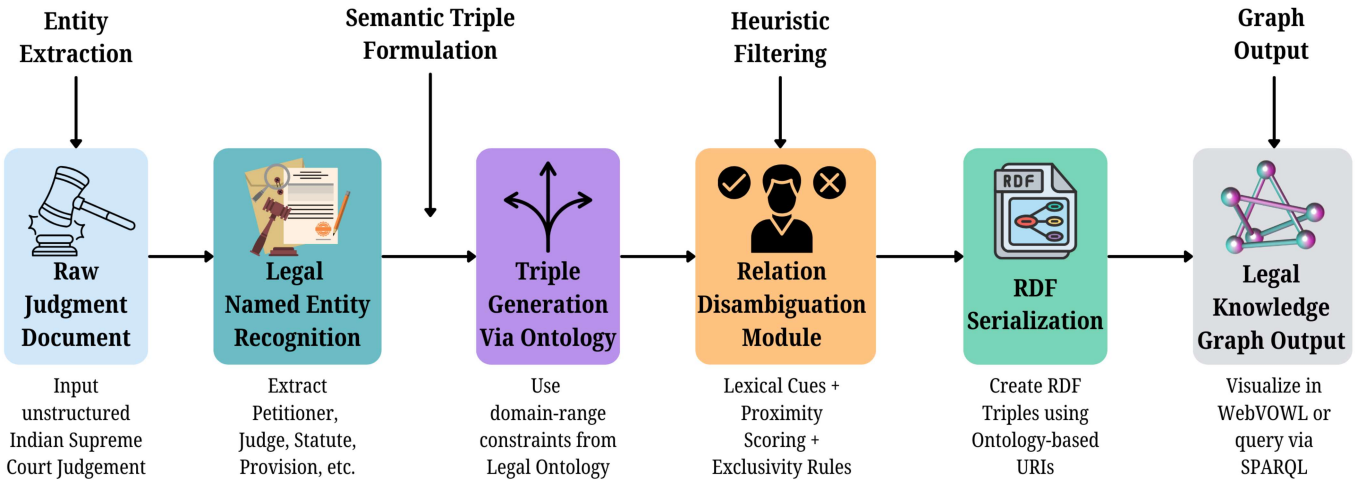

Fig. 1. Multi-stage pipeline for constructing a Legal Knowledge Graph from Indian Supreme Court judgments. The framework progresses through entity extraction, ontology-driven triple generation, heuristic-based relation disambiguation, RDF serialization, and graph-based visualization.

Algorithm 1 Ontology-Constrained Relation Extraction via Syntactic and Lexical Cues

Require: NER entity set $\mathcal{E}$, judgment text $\mathcal{D}$, ontology relation set $\mathcal{R}$
Ensure: Candidate triple set $\mathcal{T}_{\text{lex}}$
Initialize $\mathcal{T}_{\text{lex}} \leftarrow \emptyset$; load trigger patterns $\mathcal{P}$
for each sentence $s \in \mathcal{D}$ **do**
  Identify entity pairs $(e_i, e_j) \in \mathcal{E}$ in s
  for each (e_i, e_j) and relation $r \in \mathcal{R}$ such that $\text{type}(e_i) = \text{domain}(r)$ and $\text{type}(e_j) = \text{range}(r)$ **do**
    for each pattern $p \in \mathcal{P}[r]$ **do**
      if p matches s **then**
        Add triple $\langle e_i, r, e_j \rangle$ to $\mathcal{T}_{\text{lex}}$
      end if
    end for
  end for
end for
return $\mathcal{T}_{\text{lex}}$

Algorithm 2 Relation Scoring via Positional Proximity

Require: NER-tagged entity set $\mathcal{E}$, triple set $\mathcal{T}_{\text{lex}}$, document $\mathcal{D}$ (split into sentences/paragraphs)
Ensure: Filtered triple set $\mathcal{T}_{\text{prox}}$
Initialize $\mathcal{T}_{\text{prox}} \leftarrow \emptyset$
Define scoring weights: $w_{\text{same\_sentence}}$, $w_{\text{same\_paragraph}}$, w_{distance}
for each triple $\langle e_i, r, e_j \rangle \in \mathcal{T}_{\text{lex}}$ **do**
  Compute $pos(e_i), pos(e_j)$ in $\mathcal{D}$
  if same sentence **then**
    $S_r \leftarrow w_{\text{same\_sentence}}$
  else if same paragraph **then**
    $S_r \leftarrow w_{\text{same\_paragraph}}$
  else
    $S_r \leftarrow \frac{1}{1+|pos(e_i)-pos(e_j)|} \cdot w_{\text{distance}}$
  end if
  if $S_r \geq \tau$ **then**
    Add triple to $\mathcal{T}_{\text{prox}}$
  end if
end for
return $\mathcal{T}_{\text{prox}}$

*2) **Positional Proximity Scoring**:* Triples extracted via syntactic cues may still include spurious or contextually weak relations. To mitigate this, we compute a proximity-based confidence score for each triple based on the location of the participating entities in the judgment. Higher weights are assigned to entity pairs that co-occur within the same sentence or paragraph, and lower scores to those further apart. Triples exceeding a configurable proximity threshold τ are retained in a refined set $\mathcal{T}_{\text{prox}}$. This procedure is presented in Algorithm 2.

*3) **Mutual Exclusivity Filtering**:* Finally, we enforce domain-informed logical constraints by eliminating contradictory triples. For each group of triples involving the same entity pair but annotated with mutually exclusive relations (e.g., `prosecutedByLawyer` and `defendedByLawyer`), we retain only the most confident relation based on prior matching scores or context. This process ensures logical consistency and prevents contradictory edges in the final graph. The filtering step is formalized in Algorithm 3.

This multi-stage filtering framework integrates lexical signals, structural proximity, and domain logic to produce a final set of high-confidence, contextually accurate triples $\mathcal{T}_{\text{final}}$, which serves as the input to the RDF serialization and graph construction stage.

E. RDF Serialization and KG Output

Each triple in $\mathcal{T}_{\text{final}}$ is serialized into the Resource Description Framework (RDF) format, adhering to W3C standards.

Algorithm 3 Filtering Mutually Exclusive Relations

Require: Triple set $\mathcal{T}_{\text{prox}}$, exclusivity rule set $\mathcal{X}$
Ensure: Conflict-free triple set $\mathcal{T}_{\text{final}}$
Initialize $\mathcal{T}_{\text{final}} \leftarrow \emptyset$
for each conflict group $\mathcal{G} \subseteq \mathcal{T}_{\text{prox}}$ based on $\mathcal{X}$ **do**
    Rank triples in $\mathcal{G}$ using lexical/proximity scores
    Retain highest-confidence triple t^*, discard others
    Add t^* to $\mathcal{T}_{\text{final}}$
end for
for each non-conflicting triple $t \in \mathcal{T}_{\text{prox}} \setminus \cup\mathcal{G}$ **do**
    Add t to $\mathcal{T}_{\text{final}}$
end for
return $\mathcal{T}_{\text{final}}$

The subject, predicate, and object are linked using Uniform Resource Identifiers (URIs) derived from the Legal Ontology. We utilize the Turtle syntax for human-readable representation, while RDF/XML is employed for compatibility with semantic web tools and graph databases.

The resulting RDF triples are subsequently ingested into a graph database or exported for visualization using frameworks such as WebVOWL. This enables intuitive exploration of the structured graph, formulation of SPARQL queries, and support for downstream tasks including precedent recommendation, citation tracking, and legal question answering.

Figure 1 provides a high-level schematic of our multi-stage framework. It illustrates the transformation process from raw Supreme Court judgments through named entity recognition, ontology-constrained triple generation, multi-stage relation filtering, and RDF-based graph construction. The final Legal Knowledge Graph output supports advanced legal information retrieval and intelligent analytics in the Indian judicial domain.

V. Evaluation and Benchmarking

To assess the effectiveness of our proposed Legal Knowledge Graph (LKG) construction pipeline, we conduct both quantitative and qualitative evaluations. The evaluation covers: (1) manual gold standard creation using legal expert annotations, (2) performance of relation extraction using standard metrics, and (3) case-level qualitative analysis.

A. Manual Annotation and Gold Standard Creation

To construct a gold standard for benchmarking relation extraction, a corpus of 15 Indian Supreme Court judgments was selected, covering diverse domains including criminal law, civil litigation, and constitutional interpretation. The average length of each document was approximately 6,000 words, with judgments sourced from a publicly available legal repository [3].

Entity mentions were first extracted using the Legal NER model, and subsequently, two trained legal annotators manually labeled all valid binary relations among identified entities. Annotation guidelines were aligned with the 28 object properties defined in the Legal Ontology. The BRAT Rapid Annotation Tool was used to assist the annotators with entity spans and relationship labels. Inter-annotator disagreements were resolved via adjudication by a legal domain expert. Inter-annotator agreement was computed using Cohen's Kappa, yielding:

$$\kappa_{\text{entity}} = 0.89, \quad \kappa_{\text{relation}} = 0.78$$

These scores indicate strong agreement and support the reliability of the annotated corpus. The final gold standard includes 186 entity pairs and 129 valid annotated triples across the 15 documents.

[3]Legal Information Institute of India (LIIofIndia): http://www.liiofindia.org

B. Implementation Details and Hyperparameters

The Legal Named Entity Recognition model is based on a RoBERTa-large transformer backbone, augmented with a Dispersed Hierarchical Attention (DHA) mechanism and a Conditional Random Field (CRF) decoding layer. The model was trained using supervised learning on an annotated corpus of Indian Supreme Court judgments.

The experiments were conducted on an HP Z4 G4 Workstation with the following configuration: Intel Xeon W-2133 processor (3.60 GHz, 6 cores, 12 logical threads), 64 GB RAM (63.7 GB usable), and NVIDIA Quadro P5000 GPU with 16 GB VRAM. The machine runs Windows 11 Pro (version 10.0.22631, build 22631) with CUDA 12.4 and NVIDIA driver version 552.34. Virtual memory was configured with a total allocation of 133 GB, with 14.5 GB available at runtime.

The implementation utilized the Hugging Face Transformers library and PyTorch for model training, while spaCy was employed for preprocessing and dependency parsing. Ontology modeling was conducted using Protégé, and RDF serialization was implemented via the RDFLib Python package. Final graph visualizations were rendered using WebVOWL.

Key hyperparameters for training the NER model are summarized in Table I.

TABLE I
Hyperparameters used for Legal NER model training

Parameter	Value
Base Model	RoBERTa-large
Attention Mechanism	Dispersed Hierarchical Attention
Decoder Layer	Conditional Random Field (CRF)
Learning Rate	2×10^{-5}
Batch Size	16
Max Sequence Length	512
Optimizer	AdamW
Epochs	10
Warmup Steps	500
Gradient Clipping	1.0
Dropout Rate	0.1

C. Relation Extraction Evaluation

The complete pipeline was applied to the same 15 judgments, producing an initial candidate set $\mathcal{T}_{\text{cand}}$ of 287 ontology-compliant triples. After relation disambiguation and filtering, the final set $\mathcal{T}_{\text{final}}$ consisted of 140 predicted triples.

Comparing against the gold standard:

$$\text{True Positives (TP)} = 114,$$
$$\text{False Positives (FP)} = 26,$$
$$\text{False Negatives (FN)} = 15$$

To better illustrate the distribution of prediction outcomes, we visualize the comparison against the manually annotated gold standard in the form of a confusion matrix, shown in Figure 2. As is standard in open-set relation extraction tasks, we do not report True Negatives (TN), since the total number of possible negative relation instances is unbounded and not annotated. The matrix thus highlights the trade-off between precision and recall by focusing on the number of correct predictions (True Positives), spurious relations (False Positives), and missed gold relations (False Negatives).

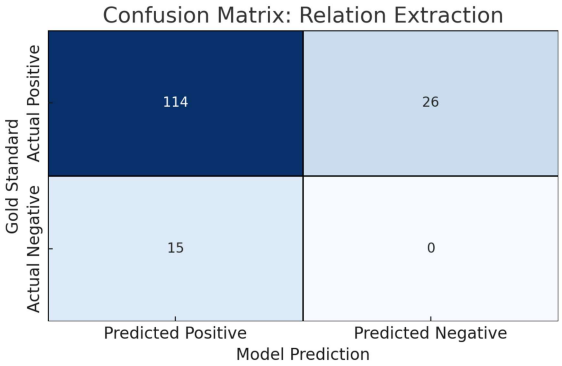

Fig. 2. Confusion matrix illustrating the performance of the relation extraction pipeline on 15 Indian Supreme Court judgments. The matrix includes only True Positives (TP), False Positives (FP), and False Negatives (FN), as True Negatives (TN) are undefined in open-set relation extraction settings.

We report the following scores:

$$\text{Precision} = \frac{|\text{Correct Triples Predicted}|}{|\text{Total Predicted Triples}|} = \frac{114}{114+26} = 0.814 \tag{2}$$

$$\text{Recall} = \frac{|\text{Correct Triples Predicted}|}{|\text{Total Gold Std. Triples}|} = \frac{114}{114+15} = 0.884 \tag{3}$$

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot 0.814 \cdot 0.884}{0.814 + 0.884} = 0.847 \tag{4}$$

These results demonstrate that the pipeline achieves high recall, as expected due to exhaustive triple generation, while maintaining reasonably strong precision through the disambiguation stages.

D. Qualitative Evaluation: Case Study

We conducted a case-level qualitative analysis using the landmark judgment *Lalita Kumari v. Government of Uttar Pradesh* (2013) 14 SCC 1. The case involved the petitioner (Lalita Kumari), the State Government, the local police department, and multiple statutory references including Section 154 of the CrPC.

Table II showcases a representative sample of predicted triples and their alignment with the annotated gold standard.

TABLE II
EXAMPLES FROM *Lalita Kumari v. Govt. of U.P.*: PREDICTED VS. GOLD STANDARD TRIPLES

Entity Pair	Predicted Relation	Correct?
Lalita Kumari → State of U.P.	`appealedAgainst Respondent`	✓
Lalita Kumari → Station House Officer	`filedComplaintWith`	✓
Judgment → Section 154 CrPC	`refersToProvision`	✓
Judge P. Sathasivam → Supreme Court	`judgeBelongsToCourt`	✓
Section 154 → Constitution of India	`refersToProvision`	×

The final incorrect relation (fifth row) demonstrates a failure of the disambiguation step, where a provision was incorrectly linked to the Constitution instead of its correct parent statute (CrPC). Such errors, although infrequent, highlight avenues for improvement in statute-provision resolution.

This comprehensive evaluation validates both the accuracy of extracted semantic relations and the interpretability of the resulting legal knowledge graph. The methodology is extensible and domain-robust, opening avenues for large-scale legal analytics, cross-case reasoning, and automated legal research support.

VI. CONCLUSION AND FUTURE WORK

In this work, we presented a comprehensive and ontology-driven pipeline for constructing a Legal Knowledge Graph (LKG) from Indian Supreme Court judgments. The proposed system integrates a fine-grained Legal Named Entity Recognition model with a formally specified Legal Ontology to produce semantically grounded triples. To ensure contextual fidelity, we introduced a multi-stage relation disambiguation module that incorporates syntactic cues, positional proximity, and mutual exclusivity constraints. Our evaluation—conducted using a manually annotated gold standard—demonstrated that the pipeline achieves high recall and competitive precision, validating both the semantic richness and structural integrity of the generated knowledge graph. A case-level analysis further illustrated the practical interpretability of the extracted triples in real-world legal discourse.

Looking ahead, we envision several directions for advancing this work. First, we aim to integrate **neural relation extraction models**, particularly those fine-tuned for legal discourse, to capture nuanced relations that may be missed by rule-based heuristics. Second, we plan to implement **cross-document coreference resolution** to enable entity and event linking across multiple judgments, thus facilitating the construction of richer, interlinked legal knowledge graphs. In terms of scalability, we will expand our corpus to include High Court judgments, which present distinct linguistic and structural challenges compared to Supreme Court rulings. Finally, we

plan to embed the constructed LKG into downstream applications such as **legal question answering**, **semantic retrieval**, and **automated legal summarization** thereby enabling intelligent access to complex legal information and supporting decision-making for both practitioners and the public.

This work serves as a foundational step toward scalable, interpretable, and semantically rigorous legal AI systems grounded in real-world judicial data from the Indian legal ecosystem.

Appendix A
Trigger Patterns for Ontology-Based Relation Extraction

Table III lists the domain-specific syntactic patterns and phrases associated with each object property in the Legal Ontology. These trigger patterns are used to support lexical matching during the relation extraction stage.

Appendix B
Mutual Exclusivity Rules for Conflict Resolution

Table IV enumerates relation pairs that are logically disjoint, along with their applicable conflict scopes and descriptions. These are used during triple pruning.

Acknowledgment

The authors would like to thank the legal domain experts involved in the annotation and evaluation process for their valuable insights and contributions. We also acknowledge the institutional support and computing infrastructure provided by the Department of Computer Science and Engineering, College of Engineering Guindy, Anna University. This work would not have been possible without the continued guidance from our academic advisors and the constructive feedback received during the development of the Legal NER and Ontology components.

References

[1] P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named entity recognition in indian court judgments," *arXiv preprint arXiv:2211.03442*, 2022.

[2] S. Lohmann, V. Link, E. Marbach, and S. Negru, "Webvowl: Web-based visualization of ontologies," in *International Conference on Knowledge Engineering and Knowledge Management*. Springer, 2014, pp. 154–158.

[3] S. Jain, P. Harde, N. Mihindukulasooriya, S. Gosh, A. Bisht, and A. Dubey, "Constructing a knowledge graph from indian legal domain corpus." in *TEXT2KG/MK@ ESWC*, 2022, pp. 80–93.

[4] M. Abdurahman, F. Darari, H. Lesmana, M. Hartopo, I. Rhesa, and B. C. L. Tobing, "Lex2kg: automatic conversion of legal documents to knowledge graph," in *2021 International Conference on Advanced Computer Science and Information Systems (ICACSIS)*. IEEE, 2021, pp. 1–5.

[5] M.-Q. Hoang, T.-M. Nguyen, H.-T. Nguyen, H.-T. Nguyen *et al.*, "Constructing a knowledge graph for vietnamese legal cases with heterogeneous graphs," in *2023 15th International Conference on Knowledge and Systems Engineering (KSE)*. IEEE, 2023, pp. 1–6.

[6] J. Li, L. Qian, P. Liu, and T. Liu, "Construction of legal knowledge graph based on knowledge-enhanced large language models," *Information*, vol. 15, no. 11, p. 666, 2024.

[7] S. Bi, Z. Ali, M. Wang, T. Wu, and G. Qi, "Learning heterogeneous graph embedding for chinese legal document similarity," *Knowledge-Based Systems*, vol. 250, p. 109046, 2022.

[8] R. Hoekstra, J. Breuker, M. Di Bello, A. Boer *et al.*, "The lkif core ontology of basic legal concepts." *LOAIT*, vol. 321, pp. 43–63, 2007.

[9] M. Palmirani, G. Governatori, A. Rotolo, S. Tabet, H. Boley, and A. Paschke, "Legalruleml: Xml-based rules and norms," in *International Workshop on Rules and Rule Markup Languages for the Semantic Web*. Springer, 2011, pp. 298–312.

[10] J. M. Schneider, G. Rehm, E. Montiel-Ponsoda, V. Rodríguez-Doncel, P. Martín-Chozas, M. Navas-Loro, M. Kaltenböck, A. Revenko, S. Karampatakis, C. Sageder *et al.*, "Lynx: A knowledge-based ai service platform for content processing, enrichment and analysis for the legal domain," *Information Systems*, vol. 106, p. 101966, 2022.

[11] F. Sovrano, M. Palmirani, and F. Vitali, "Legal knowledge extraction for knowledge graph based question-answering," in *Legal knowledge and information systems*. IOS Press, 2020, pp. 143–153.

[12] J. S. Dhani, R. Bhatt, B. Ganesan, P. Sirohi, and V. Bhatnagar, "Similar cases recommendation using legal knowledge graphs," *arXiv preprint arXiv:2107.04771*, 2021.

[13] A. Crotti Junior, F. Orlandi, D. Graux, M. Hossari, D. O'Sullivan, C. Hartz, and C. Dirschl, "Knowledge graph-based legal search over german court cases," in *European Semantic Web Conference*. Springer, 2020, pp. 293–297.

[14] H. Yu, H. Li *et al.*, "A knowledge graph construction approach for legal domain," *Tehnički vjesnik*, vol. 28, no. 2, pp. 357–362, 2021.

[15] S. A. Mukund and K. Easwarakumar, "Dispersed hierarchical attention network for machine translation and language understanding on long documents with linear complexity," in *Proceedings of the 20th International Conference on Natural Language Processing (ICON)*, 2023, pp. 90–98.

[16] M. A. Musen, "The protégé project: a look back and a look forward," *AI matters*, vol. 1, no. 4, pp. 4–12, 2015.

[17] J. Z. Pan, "Resource description framework," in *Handbook on ontologies*. Springer, 2009, pp. 71–90.

TABLE III
TRIGGER PATTERNS FOR ONTOLOGY-BASED RELATION EXTRACTION

Relation Name	**Trigger Patterns / Phrases**
hasPetitioner	"the petitioner", "filed by", "appeal by", "instituted by", "plaintiff"
hasRespondent	"the respondent", "against the respondent", "defendant", "opposite party"
hasWitness	"examined as witness", "testified", "witnessed the incident"
involvesPerson	"person involved", "individual concerned", "named individual"
judgedBy	"delivered by Justice", "judgment by", "authored by", "pronounced by", "coram"
representedByLawyer	"represented by", "appeared through counsel", "advocate for", "argued by"
prosecutedByLawyer	"prosecuting counsel", "counsel for the petitioner", "representing the appellant"
defendedByLawyer	"defense counsel", "advocate for the respondent", "appeared for the respondent"
heardAtCourt	"heard before", "in the Hon'ble Supreme Court", "in the High Court of", "appeared before"
relatedToStatute	"under the [Statute]", "provisions of", "as per the Act", "in accordance with the Act"
refersToProvision	"Article 21", "Section 302 IPC", "provision of", "under Section", "clause of"
citesPrecedent	"relied on", "cited", "referred to", "as held in", "as observed in", "in the case of"
appealsToCourt	"filed an appeal before", "approached the Supreme Court", "petitioned the court"
appealedAgainstRespondent	"appeal against", "challenged the order of", "accused/respondent"
precedentCreatedByCourt	"pronounced by", "decided by", "judgment delivered by [Court Name]"
precedentGivenByJudge	"authored by Justice", "opinion written by", "delivered by Hon'ble Justice"
organizationInvolvedIn	"represented by [Org]", "filed by [Org]", "intervenor organization"
caseFiledOnDate	"filed on", "petition dated", "case instituted on"
judgmentDeliveredOn	"judgment dated", "delivered on", "pronounced on"
caseHasCaseNumber	"Case No.", "Criminal Appeal No.", "W.P.(C) No.", "Civil Appeal No."
organizationRepresentsParty	"represented by [Org]", "legal counsel from [Org]", "advocated by [Org]"
courtHearsParty	"heard arguments of the petitioner/respondent", "appeared before the court"
partyAppearsBeforeCourt	"petitioner/respondent appeared before", "present in court"
judgeBelongsToCourt	"Justice of the Supreme Court", "High Court judge", "Justice from [Court]"
lawyerBelongsToOrganization	"from [Law Firm]", "partner at [Organization]", "legal officer at [Org]"
lawyerPracticesInCourt	"practicing in the High Court/Supreme Court", "appeared before [Court]"
organizationLocatedInGPE	"headquartered in [City]", "based in [State]", "located in [Region]"

TABLE IV
MUTUAL EXCLUSIVITY RULES FOR RELATION CONFLICT RESOLUTION

Relation 1	**Relation 2**	**Conflict Scope**	**Conflict Description**
prosecutedByLawyer	defendedByLawyer	Same Lawyer	A lawyer cannot prosecute for petitioner and defend the respondent in the same case
representedByLawyer	representedByLawyer	Same Lawyer, Different Parties	One lawyer cannot represent both petitioner and respondent simultaneously
heardAtCourt	heardAtCourt	Same Case, Multiple Courts	A single case is generally heard at only one court; conflicting mentions should be resolved
judgedBy	judgedBy	Same Case, Different Judges (single-judge bench)	If judgment is from a single judge, multiple authorship is not valid (unless explicitly stated)
appealsToCourt	heardAtCourt	Same Petitioner	A party cannot appeal to one court and be heard in another unless case is transferred
hasPetitioner	hasRespondent	Same Entity	A single person cannot be both petitioner and respondent in the same case
partyAppearsBeforeCourt	courtHearsParty	Different Courts, Same Party	A party cannot appear before two courts in the same case unless explicitly stated
precedentGivenByJudge	precedentCreatedByCourt	Conflicting Sources for Precedent	A precedent cannot be simultaneously attributed to two unrelated judges/courts
organizationRepresentsParty	representedByLawyer	Org vs. Lawyer for Same Party	If both are present, must validate org-lawyer affiliation to avoid duplication or misattribution
caseFiledOnDate	caseFiledOnDate	Different Dates	Multiple filing dates for the same case is invalid unless amended (should reconcile)
judgmentDeliveredOn	judgmentDeliveredOn	Different Dates	Multiple delivery dates for same judgment signals error